

DevOps Project

CloudOpsHub: Build an Automated Multi-Cluster Infrastructure Platform with Continuous Delivery and Observability

Background

CloudOpsHub is a mid-sized SaaS company offering analytics dashboards to SMEs across Africa and Europe. Their customers depend on uptime, data integrity, and consistent performance. As user adoption grows, CloudOpsHub faces long deployment times, inconsistent environments, and manual infrastructure provisioning that often leads to configuration drift. Recently, they have also experienced network connectivity issues causing latency for users in different regions.

To ensure easier user connectivity, high availability, and localized access, their management wants to modernize operations with an infrastructure platform that supports automated provisioning and multi-environment pipelines (dev, staging, prod) deployed across multiple Kubernetes clusters in different locations. You have been brought in as part of the DevOps and Cloud Operations team to build this automation-driven, secure, and observable multi-cluster platform.

Business Requirements

- **Infrastructure Automation:** Provision all infrastructure as code using Terraform or similar open-source IaC tools.
- **Configuration Management:** Use an open-source tool like Ansible to configure servers or services post-provisioning.
- **Multi-Cluster Environment Setup:** Build and manage at least two physically separate clusters (e.g., one local cluster and one cloud-managed cluster) with consistent configurations.
- **Centralized Continuous Delivery:** Install the Continuous Delivery (CD) tool (e.g., ArgoCD) on only one cluster. This single CD installation must manage and push deployments to all other clusters in the setup.
- **Containerized Deployment:** Deploy the analytics platform on Kubernetes, ensuring versioned and repeatable deployments.
- **Secure CI/CD Pipelines:** Integrate vulnerability scans and secrets management into pipelines.
- **Observability:** Centralized logging, metrics, and tracing for platform monitoring.
- **Resilience & Backups:** Implement recovery mechanisms and autoscaling for reliability.
- **Cost Efficiency:** Optimize compute and storage resources to avoid waste.

User Journey

Developer:

- Pushes code changes to Git.
- CI pipeline automatically builds, tests, scans, and publishes Docker images.
- CD pipeline promotes updates through staging → production across the multiple clusters via GitOps or an approval workflow.

Operations Engineer:

- Uses Terraform to provision infrastructure (multi-cluster setup, networks, monitoring stack).
- Runs Ansible to configure environments consistently.
- Monitors system metrics, manages rollbacks, and ensures uptime.

Management / Product Owner:

- Monitors dashboards for uptime and resource utilization.
- Approves production releases after testing results in staging.

Deliverables

Architecture Design & Documentation

- Present a well-defined DevOps architecture covering build, test, deployment, and monitoring stages.
- Include diagrams showing how the CI/CD pipeline, multi-cluster environments, and observability components connect.
- Justify your chosen open-source tools and design decisions.

Infrastructure Automation

- Implement automated provisioning and configuration of your multi-cluster environments.
- Include a reproducible setup for development, staging, and production.
- Show consistency in environment setup and deployment processes.

End-to-End CI/CD Pipeline

- Build a complete pipeline that automates building, testing, deployment, and rollback of the application.
- Demonstrate environment promotion from development → staging → production across the separate clusters.
- Integrate quality checks and automated notifications.

Application Deployment & Configuration

- Containerize or automate deployment of the application components (frontend, backend, database).
- Ensure environment configurations (variables, secrets, etc.) are properly managed.
- Demonstrate reliability during deployment and rollback across the different clusters.

Monitoring, Logging, and Alerts

- Implement centralized monitoring and logging across all environments.
- Show meaningful dashboards and metrics that reflect system health and performance.
- Include a mechanism for alerts or incident notifications.

Security and Compliance

- Integrate basic security checks into your DevOps workflow (e.g., scanning, secrets management, etc.).
- Demonstrate how you ensure integrity and compliance in your pipeline and infrastructure setup.
- Highlight security hardening or audit controls implemented.

Backup and Recovery Plan

- Develop and document a strategy for data backup and system recovery.
- Include procedures for restoring from a backup or rolling back a failed deployment.

Documentation and Runbooks

- Provide clear setup documentation and operational runbooks for:
- Running and maintaining the pipeline.
- Monitoring and troubleshooting deployments.
- Performing backups, restores, and rollbacks.
- The documentation should be clear enough for another team to reproduce your setup.

Demo and Presentation

- A concise technical document summarizing design, implementation, challenges, and lessons learned.
- Runbooks for core operations (deployment, rollback, monitoring setup, troubleshooting).
- 10-15 minute live presentation showing end-to-end workflow: from code commit to multi-cluster deployment and monitoring visibility.